

```

"""
simulation.py
-----
シミュレーションの本体。
1世代 = 環境刺激 → LLM生成 → アノマリー検出 → z更新 → 選択・繁殖
"""

import json
import time
import numpy as np
from pathlib import Path

from agent      import SoulAgent
from anomaly    import AnomalyDetector
from evolution  import evolve_population
from llm_client import LLMClient

def env_factor_for_generation(g: int, master_rng: np.random.Generator) -> float:
    """
    周期的環境変化（40世代周期）＋ランダムショック（20%確率）。
    """
    phase    = np.sin(2 * np.pi * g / 40)
    base     = 1.0 + 0.2 * phase
    ef       = master_rng.uniform(base * 0.9, base * 1.1)

    if master_rng.random() < 0.2:
        ef *= master_rng.uniform(0.3, 0.6)

    return float(ef)

def run_simulation(
    generations: int    = 50,
    population_size: int = 10,
    master_seed: int     = 42,
    mock: bool          = True,
    api_key: str         = None,
    output_dir: str      = "results",
    verbose: bool        = True,
) -> list[dict]:
    """
    メインのシミュレーションループ。

```

Parameters

generations : 世代数
population_size : 人口
master_seed : 再現性のためのseed
mock : True → APIを叩かずモックテキストを使用
api_key : Anthropic APIキー (mock=Falseのとき必要)
output_dir : ログ保存先ディレクトリ
verbose : 進捗をコンソールに表示するか

Returns

log : 世代ごとの記録リスト

"""

```
out_path = Path(output_dir)
out_path.mkdir(parents=True, exist_ok=True)
```

```
master_rng = np.random.default_rng(master_seed)
llm         = LLMClient(api_key=api_key, mock=mock)
detector    = AnomalyDetector()
```

初期集団

```
agents = [
    SoulAgent(
        agent_id=i,
        rng=np.random.default_rng(master_rng.integers(0, 2**31)),
    )
    for i in range(population_size)
]
```

log = []

```
for g in range(generations):
    t_start = time.time()
```

— 1. 環境刺激 —————

```
ef = env_factor_for_generation(g, master_rng)
```

```
for agent in agents:
    agent.step(ef)
```

```
mean_H = float(np.mean([a.H for a in agents]))
```

```
for agent in agents:
    agent.maybe_trigger_Z(mean_H)
```

— 2. LLM生成 —————

```

texts = []
for agent in agents:
    prompt = agent.build_prompt(ef)
    text = llm.generate(prompt)
    agent.receive_expression(text)
    texts.append(text)

# — 3. アノマリー検出 → z更新 —————
anomaly_scores = detector.score_population(texts)

for agent, score in zip(agents, anomaly_scores):
    agent.update_Z(score * 5.0) # スケーリング ([0,5] → z更新量)

# — 4. 世代統計 —————
z_agents = [a for a in agents if a.has_Z]
z_count = len(z_agents)
mean_Z = float(np.mean([a.Z for a in z_agents])) if z_agents else 0.0

# アノマリートップ個体のテキスト
top_idx = int(np.argmax(anomaly_scores))
top_agent = agents[top_idx]

gen_record = {
    "generation": g,
    "env_factor": round(ef, 4),
    "mean_H": round(mean_H, 4),
    "mean_Z": round(mean_Z, 4),
    "Z_count": z_count,
    "Z_ratio": round(z_count / population_size, 4),
    "mean_anomaly_score": round(float(np.mean(anomaly_scores)), 4),
    "max_anomaly_score": round(float(np.max(anomaly_scores)), 4),
    "top_anomaly_agent_id": top_agent.agent_id,
    "top_anomaly_text": texts[top_idx],
    "mean_stability": round(float(np.mean([a.stability for a in agents])), 4),
    "mean_alpha": round(float(np.mean([a.rule_alpha for a in agents])), 4),
    "elapsed_sec": round(time.time() - t_start, 2),
    # 全エージェントの詳細 (OSF用)
    "agents": [
        {**a.state_dict(), "anomaly_score": round(anomaly_scores[i], 4),
         for i, a in enumerate(agents)}
    ],
}

log.append(gen_record)

if verbose:
    mode = "MOCK" if mock else "API"

```

```

        print(
            f"[{mode}] Gen {g:3d} | "
            f"H={mean_H:6.2f}  Z={mean_Z:6.2f}  "
            f"Z_ratio={z_count}/{population_size}  "
            f"anomaly_max={anomaly_scores[top_idx]:.3f}  "
            f"env={ef:.3f}"
        )
        print(f"                top: \"{texts[top_idx][:80]}\"")

# — 5. 選択・繁殖 —————
agents = evolve_population(agents, population_size, master_rng)

# 結果を保存
log_path = out_path / "simulation_log.json"
with open(log_path, "w", encoding="utf-8") as f:
    json.dump(log, f, indent=2, ensure_ascii=False)

# OSF用の軽量サマリー（全テキスト除いたもの）
summary = [
    {k: v for k, v in rec.items() if k != "agents"}
    for rec in log
]
summary_path = out_path / "summary.json"
with open(summary_path, "w", encoding="utf-8") as f:
    json.dump(summary, f, indent=2, ensure_ascii=False)

if verbose:
    print(f"\n結果を保存しました: {out_path}/")

return log

```